A Building Blocks

A.1 Reliable Broadcast (RBC)

RBC is a fundamental primitive in distributed systems to ensure the consistent delivery of messages in the presence of Byzantine faults. In RBC, a sender replica p_i first broadcasts a message m in a round $r \in \mathbb{N}$ by invoking Bcast(m, r). By running the protocol, a replica p_k may output $Deliver(m, r, i)$, where m is the broadcast message, r is the round number, and i is the identifier of the sender replica. RBC guarantees the following properties:

- **Validity:** If a correct replica p_j invokes Bcast(m, r), then every correct replica p_i eventually invokes $Deliver(m, r, j)$.
- **Integrity:** A replica delivers a message m at most once, and only if m was broadcast by replica p_j via Bcast(m, r).
- **Agreement:** If a correct replica p_i invokes $Deliver(m, r, j)$, then all correct replicas eventually invoke $Deliver(m, r, j)$.

Asynchronous DAG-based protocols rely on RBC to deliver blocks, accounting for 62.3% of the latency, as shown in Sec. 2. Thus, we propose an efficient *TEE-assisted Reliable Broadcast (T-RBC)* with linear message complexity, one step of message broadcast (instead of two), and tolerance of a minority of Byzantine nodes (Sec. 5.1).

A.1.1 T-RBC Protocol Flow. Fig. 9 gives the detailed pseudocode and message pattern of T-RBC in Fides. The primary routine TBCast runs on every broadcast and, in the good case, incurs only **a single broadcast**. The auxiliary routine TCatchup incurs two additional message rounds and is triggered only when a replica observes a missing vertex in the causal history.

TEE-assisted Reliable Broadcast (T-RBC) at replica p_i

1: **function** TBCast(v) **do**
2: **Step 0: Monotonic Counter Value Generation**
3: $v.counter \leftarrow$ MC.GetCounter$(round\text{-}cert, v)$
▷ TEE generates a unique signed counter value
4: **Step 1: Broadcast**
5: As a sender, broadcast $\langle$TBCast$, v\rangle$.
6: As a receiver, deliver v upon receiving $\langle$TBCast$, v\rangle$ with a valid counter value.

7: **function** TCatchup(v', p_i) **do**
8: **Step 2: Request for the missing vertex**
9: **Upon observing** a missing vertex v' in the causal history of v from p_j, replica p_i actively sends $\langle$Request$, v'\rangle$ to p_j.
10: **Step 3: Send the vertex v'.**
11: **Upon receiving** a valid $\langle$Request$, v'\rangle$ from a replica p_i, replica p_j will then send out the full vertex $\langle$TCatchup$, v'\rangle$ which p_j requested.
12: p_i **delivers** v' upon receiving a valid $\langle$TCatchup$, v'\rangle$.

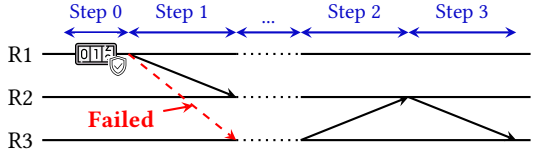

Figure 9: TEE-assisted Reliable Broadcast (*T-RBC*) with *MC*.

A.2 Common Coin

The common coin (also known as the global random coin) enables random decision-making among a set of participants, playing an essential role in asynchronous consensus [21, 24, 25]. In Fides, the common coin is utilized in every wave $w \in \mathbb{N}$. A replica invokes the coin by calling coinLeader(w) to select p_i as the leader of wave w. The common coin provides the following guarantees:

- **Agreement:** If two correct replicas invoke coinLeader(w) to obtain p_i and p_k, respectively, then $p_i = p_k$.
- **Termination:** Every correct replica invoking the function will eventually receive a result when the function has been invoked by more than f replicas.
- **Unpredictability:** If no more than f replicas have invoked coinLeader(w), the outcome is computationally indistinguishable from a random value, except with negligible probability ϵ.

Traditional common-coin implementations [65] that use resource-intensive cryptographic techniques are expensive, as indicated in Sec. 2. Thus, we leverage TEEs to realize a more efficient *TEE-assisted Common Coin (T-Coin)* (Sec. 5.3).

B Correctness Analysis of Building Blocks

B.1 TEE-assisted Reliable Broadcast (T-RBC)

Theorem A.1. *The TEE-assisted Reliable Broadcast in Fides satisfies **Validity**.*

Proof. Suppose a correct initiator p_j invokes TBCast(v) at round r. By calling the Monotonic Counter *MC* inside the TEE, p_j obtains a unique TEE-signed counter for (p_j, r), ensuring that at most one valid message can be produced for that round. Any replica delivers v upon receiving a $\langle$TBCast$, v\rangle$ accompanied by a valid *MC* certificate.

If a Byzantine adversary selectively delivers the $\langle$TBCast$, v\rangle$ message to only a subset of correct replicas, any such receiver will reference v in its next-round vertex, making v's metadata visible system-wide. By the causality property of DAG, other correct replicas will eventually observe v and thus trigger TCatchup to fetch the certified payload and then deliver v. Hence, if a correct initiator invokes TBCast(v), all correct replicas eventually $Deliver(v)$, satisfying *Validity*. □

Theorem A.2. *The TEE-assisted Reliable Broadcast in Fides satisfies **Integrity**.*

Proof. By design, *MC* enforces at-most-one valid counter per sender and round: only the authorized initiator $v.source$ can produce a unique valid counter for round $v.round$. If an adversary attempts to forge the vertex v or create multiple conflicting messages for the same round, such messages will fail the TEE validation. Because no two messages share the same counter and replicas reject uncertified messages, each vertex is delivered at most once and only if genuinely broadcast by $v.source$. This guarantees *Integrity*. □

Theorem A.3. *The TEE-assisted Reliable Broadcast in Fides satisfies **Agreement**.*

Proof. Assume some correct replica p_i has $Deliver_i(v)$. Either p_i received a valid $\langle$TBCast$, v\rangle$ directly, or it obtained v via TCatchup

after seeing v's metadata in DAG references. In both cases, v's metadata is propagated in subsequent vertices, so every correct replica will eventually observe it and invoke TCatchup to retrieve the certified payload. By Integrity, there is at most one valid payload per (source, round), so all correct replicas deliver the same v. Thus **Agreement** holds. □

B.2 TEE-assisted Common Coin (T-Coin)

Theorem A.4. *The TEE-assisted Common Coin in Fides satisfies **Agreement**.*

Proof. When at least $f+1$ replicas invoke coinLeader(w) for the same wave w, they rely on the same shared seed and input parameters within their *RNG* components. Since the *RNG* operates deterministically based on these inputs and the shared cryptographic seed, it produces the same output at all correct replicas. Thus, the selected leader is identical across all correct replicas, satisfying the **Agreement** property. □

Theorem A.5. *The TEE-assisted Common Coin in Fides satisfies **Termination**.*

Proof. If at least $f+1$ replicas invoke coinLeader(w), sufficient valid *MC* values are generated to construct the required quorum proof. The reliable broadcast mechanism ensures that these *MC* values, embedded in the corresponding vertices, are eventually delivered to all correct replicas. Consequently, any correct replica invoking coinLeader(w) can collect the necessary quorum proof and obtain a response from the *RNG* component, thus ensuring **Termination**. □

Theorem A.6. *The TEE-assisted Common Coin in Fides satisfies **Unpredictability**.*

Proof. As long as fewer than $f+1$ replicas have invoked coinLeader(w), the quorum proof required by the *RNG* component cannot be formed. Without this proof, the output of the *RNG* remains inaccessible and unpredictable to any adversary. Additionally, the internal state of the *RNG* is securely protected within the TEE, preventing external entities from predicting its output. Therefore, the outcome remains computationally indistinguishable from random, satisfying the **Unpredictability** property. □

C Correctness Analysis for Reliable Dissemination

Claim 1. *If a correct replica p_i adds a vertex v to its DAG, then eventually, all correct replicas add v to their DAGs.*

Proof. We prove the claim by induction on the rounds.

Base Case: In the initial round, all correct replicas share the same initial state, so the claim holds trivially.

Inductive Step: Assume that for all rounds up to $r-1$, any vertex added by a correct replica is eventually added to the DAGs of all other correct replicas.

Now consider a vertex v added by a correct replica p_i in round r. When v is reliably delivered to p_i (line 7), p_i conducts validation, including verifying v's validity (line 8) and ensuring all referenced vertices by v are already in the *DAG* (line 9).

By the *Agreement* property of reliable broadcast (Sec. A.1), v will eventually be delivered to all correct replicas. Since v passed these checks at p_i, and all correct replicas have consistent DAGs for earlier rounds by the induction hypothesis, v will pass validation at all correct replicas. Thus, all correct replicas will eventually add v to their DAGs. □

Lemma 2. *All correct replicas will eventually have an identical DAG view.*

Proof. From Claim 1, any vertex added by a correct replica will eventually be added to the DAGs of all other correct replicas. Despite network asynchrony and variations in message delivery times, Claim 1 guarantees that all correct replicas receive the same set of messages. Since all correct replicas apply the same deterministic rules to incorporate these vertices into their DAGs, the DAGs of all correct replicas will converge to the same state over time. Thus, all correct replicas will eventually have identical DAG views. □

D Correctness Analysis for Fides

D.1 Safety Proof

Lemma 3. *When electing the leader vertex of wave w, for any two correct replicas p_i and p_j, if p_i elects v_i and p_j elects v_j, then $v_i = v_j$.*

Proof. To prove that $v_i = v_j$, we need to ensure that all correct replicas use the same inputs when calling the random function *RNG* (e.g., the vertices from the corresponding round in line 44) to elect the leader of the wave w. From Lemma 2, we know that all correct replicas will eventually have identical DAGs. When a correct replica attempts to elect the leader for wave w, it does so after it has advanced to the necessary round and has incorporated all required vertices from previous rounds. Since the DAGs are identical and the random seed in *RNG* is the same across replicas, the inputs to the *Rand* function will be identical. Therefore, the output p_j selected as the leader will be the same at all correct replicas. Consequently, the vertex v proposed by p_j in round w will be the same at all correct replicas, so $v_i = v_j$. □

Lemma 4. *If the leader vertex v of wave w is committed by a correct replica p_i, then for any leader vertex v' committed by any correct replica p_j in a future wave $w' > w$, there exists a path from v' to v.*

Proof. A leader vertex v in wave w is committed only if at least $f+1$ third-round vertices in wave w reference v. Since each vertex in the DAG must reference at least $f+1$ vertices from the previous round, quorum intersection ensures that all vertices in the first round of wave $w+1$ have paths to v. By induction, this property extends to all vertices in all rounds of waves $w' > w$. Thus, every leader vertex v' in waves $w' > w$ has a path to v, proving the lemma. □

Lemma 5. *A correct replica p_i commits leader vertices in ascending order of waves.*

Proof. In the algorithm, when a leader vertex v of wave w is committed (either directly or indirectly), the replica pushes v onto a stack (line 36 or line 40). The algorithm then recursively checks for earlier uncommitted leader vertices in waves w' from $w-1$ down to the last decided wave (line 37). If such a leader v' exists

and there is a path from v to v', v' is also pushed onto the stack. Since the stack is a LIFO (Last-In-First-Out) structure when popping vertices to commit (Line 50), the leaders are committed in order from the earliest wave to the current wave w. This ensures that leader vertices are committed in ascending order of waves. □

Lemma 6. *All correct replicas commit leader vertices in the same ascending order of waves.*

Proof. We assume that a correct replica p_i commits leader vertex v of wave w and another correct replica p_j commits leader vertex v' of wave w', where $w < w'$. By Lemma 4, there must exist a path from v' to v in the DAG. This path implies that v should have been indirectly committed by p_j. Thus, if a leader vertex is committed by one correct replica, then no other replica can skip committing it.

Combining this observation with Lemma 5, all correct replicas commit leader vertices in the same ascending order of waves. □

Theorem 3. *Fides guarantees **Safety** property.*

Proof. The commit procedure, as defined in the function OrderVertices (lines 48–54), is responsible for committing vertices.

By Lemma 6, all correct replicas commit leader vertices in the same ascending order of waves. Furthermore, each committed leader's causal history is delivered in a predefined, deterministic order (line 52). This ensures that the sequence of vertices—and, by extension, the sequence of transactions contained within these vertices—is identical for all correct replicas.

Additionally, Lemma 2 establishes that all correct replicas maintain identical DAGs. Since all correct replicas apply the same deterministic commit rules, the assignment of sequence numbers sn to transactions is consistent across all replicas. Consequently, if two correct replicas p_i and p_j commit transactions tx_1 and tx_2 with sequence number sn, it follows that $tx_1 = tx_2$. Thus, Fides satisfies the **Safety** property. □

D.2 Liveness Proof

D.2.1 Preliminaries (aligned with Sec. 7.1.1). We adopt the k-iteration common-core abstraction of Sec. 7.1.1. Let $N{=}2f{+}1$ replicas with up to f Byzantine. In a wave of k rounds, each replica i outputs a set U_i after k aggregation rounds, and the *common core* is

$$S^*(k) \;:=\; \bigcap_{i\in\{1,\dots,N\}} U_i.$$

At the end of the wave, Fides invokes a *T-Coin* to elect a leader from the first round. Let $\mathcal{L}$ denote the leader domain. We consider a *global* coin that elects uniformly from all N IDs (replicas). A wave commits if the elected leader lies in $S^*(k)$. Hence, the per-wave success probability is

$$p_k \;=\; \frac{|S^*(k)|}{|\mathcal{L}|} \qquad (|\mathcal{L}| = N).$$

Therefore the expected commitment latency (in *rounds*) is

$$\mathbb{E}[\text{rounds}] \;=\; k\cdot\frac{1}{p_k} \;=\; k\cdot\frac{|\mathcal{L}|}{|S^*(k)|}. \tag{1}$$

We keep the DAG-side notation as follows: round sets $R_1,\dots,R_\ell$, parent sets $P_{t+1}(\cdot)$ of size $f{+}1$, and for $x\in R_1$ the carrier sets $U_x^{(t)}$ and final support $supp_\ell(x) := |\{z\in R_\ell :\ P_\ell(z)\cap U_x^{(\ell-2)}\neq\emptyset\}|$.

D.2.2 DAG ⇒ Common-Core (bridge). Set $k=\ell$. If a first-round vertex $x\in R_1$ is supported by at least $f{+}1$ final-round (l-th) vertices in the DAG, i.e., $supp_\ell(x)\geq f{+}1$, then x appears in at least $f{+}1$ replicas' k-round aggregate U_i; hence

$$x\in S^*(k) \;:=\; \bigcap_{i\in\{1,\dots,N\}} U_i.$$

Therefore, any DAG-side lower bound on $\#\{x\in R_1 :\ supp_\ell(x)\geq f{+}1\}$ lifts verbatim to a lower bound on $|S^*(k)|$, and the per-wave success probability with a global random coin is $p_k = |S^*(k)|/N$. In what follows, we work entirely in the DAG view (via $supp_\ell(\cdot)$ and the constructions) to present counting arguments and adversarial schedules; by the bridge above, these results directly yield the required common-core properties and liveness bounds.

D.2.3 Proof by DAG. Before proceeding with the proof, we summarize the key symbols used throughout.

- ℓ: the number of rounds in a single wave.
- R_i: the disjoint vertex sets in round-i of the DAG. By DAG construction rules, $R_i\in[f{+}1,2f{+}1], for\ i\in\{1,\dots,\ell\}$.
- n_i: the number of vertices in round i ($n_i = |R_i|, for\ i\in\{1,\dots,\ell\}$).
- $P_{t+1}(w)\subseteq R_t$: The parent (referenced) set of $w\in R_{t+1}$, with $|P_{t+1}(w)| = f{+}1$ (all parents are distinct).
- $U_x^{(t)}$: the round-t carrier set of $x\in R_1$, *i.e.*, the set of round-t vertices that support x. Define recursively as:

$$U_x^{(2)} := \{u\in R_2 :\ x\in P_2(u)\}$$
$$U_x^{(t)} := \{w\in R_t : P_t(w)\cap U_x^{(t-1)}\neq\emptyset\}\ \ (t>2).$$

- $supp_\ell(x)$: The number of final-round (l-th) vertices that support (link to) $x\in R_1$. For ℓ rounds, $supp_\ell(x) := |\,\{z\in R_\ell : P_\ell(z)\cap U_x^{(\ell-1)}\neq\emptyset\}\,|$.
- *Heavy / Light* vertex at round 1: a vertex $x\in R_1$ is *heavy* if $|U_x^{(2)}|\geq n_2{-}f$ (equivalently, $supp(x)=n_l$), and *light* otherwise ($|U_x^{(2)}|\leq m{-}f{-}1$). We write $H := \{x\in R_1 : |U_x^{(2)}|\geq n_2-f\}$ and $L := R_1\setminus H$.

Adversarial Network Model (Worst Case). We first prove the adversarial cases.

Lemma 7. *In a **two-round** wave ($\ell=2$) with $N{=}2f{+}1$ replicas, **no** first-round vertex is guaranteed to reach $supp_2(x)\geq f{+}1$ whenever $f\geq 2$ under an asynchronous adversary.*

Proof. Assume the asynchronous adversary exists. In each round, consider any set S of $f{+}1$ vertices in round 2 with $n_1\in[f{+}1,2f{+}1]$. Hence, the total number of links from S to the first-round vertices is $(f+1)\cdot(f+1)$.

However, if we suppose each first-round vertex receives at most f incoming links from S, the aggregate capacity would be $n_1\cdot f$. The difference, $(f+1)^2-n_1\cdot f$, evaluates to $-f^2+f+1$ in the worst-case scenario ($n_1=2f+1$). Since this value is non-positive for all $f\geq 2$, the adversary can strategically construct a DAG where every vertex in round 1 has an indegree from S of at most f. Consequently, under

adversarial scheduling, the protocol may be forced to stall in every wave, as no vertex $x \in R_1$ can satisfy the required support threshold $supp_2(x) \geq f+1$. □

Lemma 8. *When employing a **two-round** commit strategy, liveness cannot be guaranteed.*

Proof. By Lemma 7, under an asynchronous adversary and $f \geq 2$, every $x \in R_1$ satisfies $supp_2(x) \leq f$, so no vertex reaches the threshold $f+1$. By the DAG⇒Common-Core bridge, this implies $S^*(2) = \emptyset$.

From the common-core preliminaries, a wave commits iff the coin-elected leader lies in $S^*(k)$; hence, for $k = 2$ the per-wave success probability is $p_2 = \frac{|S^*(2)|}{|\mathcal{L}|} = 0$ (Eq. (1)), yielding unbounded expected commit latency. An asynchronous adversary can realize this in every wave, so progress cannot be guaranteed. Therefore, liveness does not hold under a **two-round** commit strategy. □

Lemma 9. *In any wave with $\ell \geq 3$, every **heavy** vertex $x \in R_1$ satisfies $supp_\ell(x) = n_\ell \geq f+1$.*

Proof. For $x \in R_1$ put $d_x := |U_x^{(2)}|$, *i.e.*, the number of round-2 vertices linked to x. Call x *heavy* if $d_x \geq n_2 - f$; then $|R_2 \setminus U_x^{(2)}| \leq f$, so every $(f+1)$-subset of R_2 intersects $U_x^{(2)}$.

Consider any $y \in R_3$. Since $|P_3(y)| = f+1$ and $P_3(y) \subseteq R_2$, we have $P_3(y) \cap U_x^{(2)} \neq \emptyset$, hence $y \in U_x^{(3)}$. Therefore $U_x^{(3)} = R_3$ and $supp_3(x) = n_3 \geq f+1$.

For any $t \geq 3$, if $U_x^{(t)} = R_t$, then for any $z \in R_{t+1}$ we have $P_{t+1}(z) \subseteq R_t$, so trivially $P_{t+1}(z) \cap U_x^{(t)} \neq \emptyset$ and $z \in U_x^{(t)}$. By induction, $U_x^{(t+1)} = R_{t+1}$ for all $t = 3, \ldots, \ell$. Thus $supp_\ell(x) = |R_\ell| = n_\ell \geq f+1$. □

Lemma 10. *In any **three-round** wave semantics ($\ell = 3$), at least two vertices $x \in R_1$ are* heavy *and satisfy $supp(x) \geq f+1$. Moreover, this lower bound is attained when $n_1 = n_2 = 2f+1$ (for any $n_3 \in [f+1, 2f+1]$).*

Proof. For $x \in R_1$, put $d_x := |\{u \in R_2 : x \in P_2(u)\}|$ and call x *heavy* if $d_x \geq n_2 - f$. By Lemma 9, any *heavy* x satisfies $supp_3(x) = n_3 \geq f+1$.

Let H be the set of *heavy* vertices. Since each $u \in R_2$ has $f+1$ parents,

$$\sum_{x \in R_1} d_x = \sum_{u \in R_2} |P_2(u)| = n_2(f+1).$$

With $d_x \leq n_2$ for *heavy* vertices and $d_x \leq n_2 - f - 1$ for **light**,

$$n_2(f+1) \leq |H| \cdot n_2 + (n_1 - |H|) \cdot (n_2 - f - 1) = n_1(n_2 - f - 1) + |H|(f+1),$$

thus we have

$$|H| \geq n_1 + n_2 - \frac{n_1 n_2}{f+1}.$$

For fixed $n_2 \in [f+1, 2f+1]$, the derivative of n_1 equals $1 - \frac{n_2}{f+1} < 0$, hence the right-hand side is minimized at the largest n_1, *i.e.*, $n_1 = 2f+1$. Plugging $n_1 = 2f+1$ yields

$$|H| \geq (2f+1) - \frac{f n_2}{f+1},$$

which is strictly decreasing in n_2, so the minimum over $n_2 \in [f+1, 2f+1]$ is attained at $n_2 = 2f+1$:

$$|H| \geq (2f+1) - \frac{f(2f+1)}{f+1} = 2 - \frac{1}{f+1}.$$

Taking ceilings gives $|H| \geq 2$ for all admissible (n_1, n_2), and the *minimum* is attained at $(n_1, n_2) = (2f+1, 2f+1)$ (independently of n_3). □

Lemma 11. *In the **three-round** wave semantic ($\ell = 3$), exactly two vertices in the first round are guaranteed to reach $supp_3(x) \geq f+1$ under an asynchronous adversary.*

Proof. **Lower bound.** From Lemma 10, we get the lower bound of $|H| \geq 2$ for any set of (n_1, n_2, n_3). Therefore, in every legal construction, at least two $x \in R_1$ reach $supp(x) = f+1$.

Tightness via explicit construction. We next exhibit a legal construction with exactly two vertices reaching the threshold.

Step 1 (choose P_2). Pick two distinct *heavy* vertices $h_1, h_2 \in R_1$ and enforce

$$\{h_1, h_2\} \subseteq P_2(u), \quad \forall u \in R_2.$$

Thus $|U_{h_1}^{(2)}| = |U_{h_2}^{(2)}| = |R_2| = 2f+1 \geq f+1$. Let $L := R_1 \setminus \{h_1, h_2\}$ be the set of the remaining $2f-1$ **Light** vertices. For each $u \in R_2$ there remain exactly $f-1$ light-parent slots to fill so that $|P_2(u)| = f+1$. We will fill these using only a prescribed family of $f+1$ *windows* on R_2 of size f each, so that every light $x \in L$ is assigned to a window and all its descendants lie inside that window. Formally, label R_2 cyclically as $\{0, 1, \ldots, 2f\}$ and define

$$S_i := \{\, i, i+1, \ldots, i+f-1 \,\} \pmod{2f+1}, \quad i \in \{0, 2, 4, \ldots, 2f\}$$

($f+1$ windows, each of size f). Choosing the window in this way ensures that every column $u \in R_2$ is contained in either $\lfloor f/2 \rfloor$ or $\lceil f/2 \rceil$ of the windows S_i, yielding near-uniform coverage. This makes it easy to set the fractional column sums to f or $f-1$ and lift to an integral solution by total unimodularity later.

Further, we partition L into $f+1$ groups G_i so that $|G_i| = 2$ for $i = 0, 2, \ldots, 2f-4$ (that is, for $f-2$ windows), $|G_i| = 1$ for the remaining three windows; this uses all $2f-1$ light vertices. Assign each light $x \in G_i$ to a target degree $d_x \in \{f, f-1\}$ as follows: for every window G_i with two light vertices, both lights get $d_x = f$; among the three windows with only one light vertex, two of them get $d_x = f$ and one gets $d_x = f-1$. Then

$$\sum_{x \in L} d_x = (f-2) \cdot 2f + 2 \cdot f + (f-1) = (2f+1)(f-1),$$

which equals the total number of light descendants required across R_2.

Now we have a bipartite b-matching system, and we define variables $a_{x,u} \in \{0, 1\}$ for $x \in L, u \in R_2$ with the constraints

$$\begin{cases} \sum_{u \in R_2} a_{x,u} = d_x & (\forall x \in L), \\ \sum_{x \in L} a_{x,u} = f - 1 & (\forall u \in R_2), \\ a_{x,u} = 0 & \text{if } u \notin S_{i(x)}, \\ a_{x,u} \in \{0, 1\} \end{cases} \tag{2}$$

where $i(x)$ is the index of the window that x is assigned to.

By the previous rule, the vertices in R_2 linked to any $x \in R_1$ are designated by $S_{i(x)}$, thus consecutive. The constraint matrix of (2) has the ***consecutive-ones*** property on rows (each row has 1's only within the interval $S_{i(x)}$), hence it is **totally unimodular** (see [86]). Therefore, any feasible fractional solution is integral; in particular, the system admits an integer solution whenever it admits a fractional one. A fractional solution is immediate: for each $x \in L$ put $a_{x,u} = d_x/f$ for $u \in S_{i(x)}$ and $a_{x,u} = 0$ otherwise. Then row-sums are d_x by construction. For the column-sums, by suitably permuting which windows are the two-light vs. one-light window, one achieves that for every u the total $\sum_{x:u\in S_{i(x)}} d_x = f(f-1)$, hence the fractional column-sum equals $\sum_x a_{x,u} = (1/f)\cdot f(f-1) = f-1$. By *total unimodularity*, there is an *integral* solution to (2). Taking $P_2(u)$ to be $\{h_1, h_2\} \cup \{x \in L : a_{x,u} = 1\}$ for each u completes Step 1. Thus for every light x we have $|U_x^{(2)}| = d_x \le f$ and, moreover, $U_x^{(2)} \subseteq S_{i(x)}$ is confined to a window of size f.

Step 2 (choose P_3). We first consider the worst but admissible case of having only $f+1$ vertices in R_3. Use the $f+1$ vertices of R_3 to index the $f+1$ windows (S_i): for each $i \in \{0, 2, \ldots, 2f\}$ pick a distinct $v_i \in R_3$ and set

$$P_3(v_i) \;:=\; R_2 \setminus S_i. \qquad (\textbf{Note}: |P_3(v_i)| = 2f+1-f = f+1),$$

Verification. For the two *heavy* vertices h_1, h_2 we have $|U_{h_j}| = 2f+1 \ge f+1$, hence

$$supp_3(h_j) = |R_3| = f+1.$$

For any light $x \in G_i$, we have $U_x^{(2)} \subseteq S_i$ by Step 1, hence $P_3(v_i) \cap U_x^{(2)} = \emptyset$. Therefore x is *not* hit by some v_i and

$$supp_3(x) \;\le\; |R_3| - 1 \;=\; f.$$

Thus exactly only the two vertices h_1, h_2 satisfy $supp_3(\cdot) = f+1$.

Therefore, combining the lower and tightness yields exactly two vertices in the first round that are guaranteed to reach $supp_3(\cdot) \ge f+1$, which completes the proof. □

Lemma 12. *When employing a **three-round** commit strategy ($\ell = 3$), Fides commits a vertex leader every $3f+2$ rounds in the DAG under an asynchronous adversary in expectation.*

Proof. Consider any wave w. By Lemma 11, at least **two** $x \in R_1$ satisfy $supp_2(x) \ge f+1$. The common coin (Sec. A.2) ensures that the adversary cannot predict the coin's outcome before the leader is revealed. By the DAG⇒Common-Core bridge, this implies $|S^*(3)| = 2$. From the common-core preliminaries, a wave commits iff the coin-elected leader lies in $S^*(k)$; hence for $k = 3$ the per-wave success probability is $p_3 = \frac{|S^*(3)|}{|\mathcal{L}|} = \frac{2}{2f+1}$. By Eq. (1), the committed latency is

$$\mathbb{E}[\text{rounds}] \;=\; k \cdot \frac{1}{p_k} \;= 3 \cdot \frac{2f+1}{2} \;= 3f + \frac{3}{2}.$$

Therefore, in expectation, when employing a **three-round** commit strategy, Fides commits a vertex leader every $\lceil 3f + \frac{3}{2} \rceil = 3f+2$ rounds in the DAG. □

Lemma 13. *When employing a **four-round** wave semantic ($\ell = 4$), at least $f+1$ vertices $x \in R_1$ are guaranteed to satisfy $supp_4(x) \ge f+1$.*

Proof. For any $y \in R_3$ we define its *gap* as $S(y) := R_2 \setminus P_3(y)$, so $|S(y)| = f$.

Step 1 (a single protected class consumes $f+1$ third-layer vertices). Say that y *avoids* x iff $U_x^{(2)} \subseteq S(y)$. For a fixed window $S \subseteq R_2$ of size f, define the class

$$C_2(S) := \{x \in R_1 : U_x^{(2)} \subseteq S\}.$$

All $x \in C_2(S)$ are avoided exactly by those y with $S(y) = S$.

To make any x *eligible* to be avoided by some $z \in R_4$, we must have $|R_3 \setminus U_x^{(3)}| \ge f+1$, *i.e.*

$$A_x^{(3)} \;:=\; |\{y \in R_3 : P_3(y) \subseteq R_2 \setminus U_x^{(2)}\}| \;\ge\; f+1. \qquad (3)$$

Crucially, these $f+1$ avoiders cannot be combined across different gaps: to cover the entire class $C_2(S)$, one needs $f+1$ vertices $y \in R_3$ all with the *same* gap S. Since $|R_3| \le 2f+1 < 2(f+1)$, at most one window $S^\star$ can receive $\ge f+1$ such vertices; hence at most one class $C_2(S^\star)$ can be fully protected (*i.e.*, every $x \in C_2$ is possible to be avoided by (not linked to) vertex in R_4).

*Step 2 (there are **$f+1$** vertices outside any single window).* Fix any size-f set $S \subseteq R_2$ and consider the columns $R_2 \setminus S$. Each $u \in R_2 \setminus S$ must choose $f+1$ parents in R_1, so the total number of *outside-S* parent edges equals $(n_2 - f)(f+1)$. Every $x \in C_2(S)$ has $U_x^{(2)} \subseteq S$ and thus contributes *zero* edge outside S. Thus, all these "outside-S" parent selections must be drawn from vertices outside $C_2(S)$, so the number of distinct vertices outside $C_2(S)$ satisfies:

$$|\{x \in R_1 : U_x^{(2)} \not\subseteq S\}| \;\ge\; \frac{(n_2 - f)(f+1)}{n_2 - f} \;=\; f+1. \qquad (4)$$

Therefore, there must be at least $f+1$ R_1 vertices outside $C_2(S)$.

Step 3 (outside vertices cannot be protected and must be hit by round 4). Choose the third layer so that all its gaps equal one fixed window $S^\star$ (*i.e.* $S(y) = S^\star$ for all $y \in R_3$). Then the class $C_2(S^\star)$ has $A_x^{(3)} = |R_3| \ge f+1$, hence (3) holds for all $x \in C_2(S^\star)$. By Step 1, no other class can satisfy (3). In particular, every $x \notin C_2(S^\star)$ has $A_x^{(3)} \le f$, so $|R_3 \setminus U_x^{(3)}| \le f$ and *no* vertex $z \in R_4$ can avoid x (since $P_4(z)$ must have size $f+1$). Consequently, all $z \in R_4$ hit every $x \notin C_2(S^\star)$ and $supp_4(x) = |R_4| = n_4 \ge f+1$. Applying (4) to $S^\star$ guarantees at least $f+1$ such x, proving the lower bound.

Step 4 (tightness). Since we have at most f Byzantine replicas, it is possible for them to make $n_1 = f+1$. Then, every $u \in R_2$ must choose all vertices of R_1, so each $x \in R_1$ is *heavy* and by Lemma 9 we have $supp_4(x) = n_4 \ge f+1$. Thus, the lower bound is attained.

This proves the lemma: in the **four-round** wave semantic ($\ell = 4$), at least $f+1$ vertices $x \in R_1$ are guaranteed to satisfy $supp_4(x) \ge f+1$. □

Lemma 14. *When employing a **four-round** commit strategy ($\ell = 4$), Fides commits a vertex leader in at most **8** rounds in the DAG under an asynchronous adversary in expectation.*

Proof. Consider any wave w. By Lemma 13, at least $f+1$ first-round vertices satisfy $supp_4(x) \ge f+1$. By the DAG⇒Common-Core bridge, this gives $|S^*(4)| \ge f+1$. From the common-core preliminaries, a wave commits iff the coin-elected leader lies in

$S^*(k)$; hence for $k = 4$ the per-wave success probability is $p_4 = \frac{|S^*(4)|}{|\mathcal{L}|} \geq \frac{f+1}{2f+1}$.. By Eq. (1), the committed latency is

$$\mathbb{E}[\text{rounds}] = k \cdot \frac{1}{p_k} = 4 \cdot \frac{2f+1}{f+1} \leq 8.$$

therefore, in expectation, when employing a **four-round** commit strategy, Fides commits a vertex leader every **8** rounds in the DAG. □

Lemma 15. *Fix N=2f+1 in any admissible DAG. For any wave length $\ell \geq 4$, in the worst case*

$$\left|\{ x \in R_1 : \text{supp}_I(x) \geq f+1 \}\right| = f+1.$$

Proof. (*Upper bound.*) In the worst case the adversary controls f replicas and can suppress all of their round-1 proposals, so $n_1 \in [f+1,\ 2f+1]$ can be driven down to $|R_1| = f+1$. Consequently,

$$\left|\{ x \in R_1 : \text{supp}_I(x) \geq f+1 \}\right| \leq n_1 = f+1,$$

so at most f+1 first-round vertices can reach the threshold (at round ℓ) under this worst case.

(*(Tightness at $\ell = 4$ and persistence for $\ell > 4$).*) By Lemma 13, in any $\ell = 4$ wave we *always* have at least f+1 vertices $x \in R_1$ with $supp_4(x) \geq f+1$. Combining with the upper bound above yields equality at $\ell = 4$:

$$\left|\{ x \in R_1 : supp_4(x) \geq f+1 \}\right| = f+1 \quad \text{(worst case)}.$$

Any wave semantics with more than four rounds ($\ell > 4$) yields at least as many qualifying vertices per wave, i.e., $|S^*(\ell)| \geq f+1$. However, an adversary can always force $n_1 = f+1$ independently of ℓ, so no larger worst-case guarantee is achievable for any $\ell > 4$. Therefore, the worst-case cap remains f+1 for all $\ell \geq 4$. □

Lemma 16. *Consider waves of fixed constant length ℓ in a DAG-BFT protocol with n=2f+1 replicas, where each round-$r \geq 2$ vertex references exactly f+1 parents in round $r-1$. Then $\ell = 4$ is optimal among constant-round wave designs: it is the smallest constant that guarantees constant-round* 8 *commitment, and any $\ell > 4$ does not improve the worst-case guarantee.*

Proof. By Lemmas 8 and 12, waves of length 2 or 3 can not guarantee constant-round commitment. Further, by Lemma 14, waves of length 4 *do* guarantee constant-round commitment (in particular, 8 rounds in expectation in the worst case).

Moreover, by Lemma 15, for any $\ell > 4$ the adversary can still only force $|R_1| = f+1$, so the worst-case number of round-1 vertices that can reach the threshold is capped by f+1. Increasing ℓ therefore does not improve the worst-case guarantee but only adds commit latency.

Hence $\ell = 4$ is the optimal constant length of wave that guarantees constant-round commitment under an n=2f+1 system. □

Random-Delay Network Model (Stochastic Case). We next prove the liveness of Fides within the random-delay network model.

Lemma 17 (Commit probability under random delays)**.** *Let N=2f+1 =: n and consider a wave of length $\ell = 4$ with $n_1 = n_2 = n_3 = n_4 = n$. Assume each round-$r$ vertex independently chooses exactly f+1 parents uniformly at random from R_{r-1}. Then*

$$p_4(f) \geq 0.9419, \quad \text{for all } f \geq 1,$$

with $p_4(f)$ strictly increasing in f and $p_4(f) \to 1$ as $f \to \infty$. Consequently, the expected rounds-per-commit satisfies

$$\mathbb{E}[\text{rounds per commit}] = \frac{4}{p_4(f)} < 4.25,$$

and $\mathbb{E}$[rounds per commit] $\to 4$ rapidly as f grows.

Proof. By symmetry of the coin, fix the leader $v \in R_1$. A wave commits iff this v satisfies the round-4 support threshold $supp_4(v) \geq f+1$, which (by the DAG⇒common-core bridge) is equivalent to $v \in S^*(4)$.

Round 2. Each $u \in R_2$ includes v among its f+1 parents with probability $p_1 = \frac{f+1}{n}$. Let $X := \#\{u \in R_2 : v \in P_2(u)\} \sim \text{Bin}(n, p_1)$.

Round 3. Conditioned on $X = k$, a given $y \in R_3$ hits at least one of those k "good" R_2 vertices with

$$q_3(k) = 1 - \frac{\binom{n-k}{f+1}}{\binom{n}{f+1}},$$

hence we define $T := |S_3(v)|$, the number of Round 3 vertices that link to v. Conditional on $X = k$, the random variable T follows a binomial distribution with parameters $(n, q_3(k))$; *i.e.*

$$T \mid X = k \sim \text{Bin}\big(n,\ q_3(k)\big).$$

Round 4. Conditioned on $T = t$, there are t "good" vertices in R_3 (those in $S_3(v)$). A vertex $z \in R_4$ chooses f+1 parents uniformly from R_3; it avoids $S_3(v)$ with probability $\binom{n-t}{f+1}/\binom{n}{f+1}$, hence it hits $S_3(v)$ with

$$q_4(t) = 1 - \frac{\binom{n-t}{f+1}}{\binom{n}{f+1}}.$$

Since different $z \in R_4$ choose independently, the number $Y := \#\{z \in R_4 : z \rightsquigarrow v\}$ satisfies

$$Y \,\big|\, T = t \ \sim\ \text{Bin}\big(n,\ q_4(t)\big).$$

Commit event and total probability. The four-round commit rule is $Y \geq f+1$. Taking the law of total probability over X and T yields exactly the stated triple sum $p_4(f) = \Pr[Y \geq f+1]$. In this random-delay model, the expected commit time is written as: $\mathbb{E}[\text{rounds}] = 4/p_4(f)$.

For small f, evaluating $p_4(f)$ yields

$$p_4(1) \approx 0.9419, \quad p_4(2) \approx 0.9869, \quad p_4(3) \approx 0.9970, \ldots$$

so the corresponding expected rounds per commit are $4/p_4(f) \approx 4.247,\ 4.053,\ 4.012$, respectively. Moreover, $p_4(f) \to 1$ rapidly as f grows, implying that one wave (four rounds) suffices in expectation to commit.

This completes the proof. □

How this ties back to common-core latency. By the DAG ⇒ Common-Core correspondence (Sec. 7.1.1), the event "coin-elected leader v satisfies $supp_4(v) \geq f+1$" is exactly "$v \in S^*(4)$". Thus the per-wave commit probability is $p_4 = \Pr[v \in S^*(4)]$, and the expected commit

latency formula from Eq. (1) specializes to $\mathbb{E}[\text{rounds}] = 4/p_4$ under the random-delay model.

System-Wide Liveness.

Lemma 18. *Every vertex v in the DAG will eventually be committed by every correct replica.*

Proof. When a correct replica creates a new vertex, it establishes strong edges pointing to vertices from the previous round (line 21) and weak edges pointing to vertices with no alternative path in the DAG (line 24). This construction ensures that all vertices are reachable and eventually included in the total order.

By Theorem 3, all correct replicas commit the leader vertices, along with their corresponding causal history, in the same order. Furthermore, Lemma 16 and Lemma 17 demonstrate that Fides guarantees vertices are committed within a constant number of rounds, ensuring they are eventually committed with probability 1.

Thus, every correct replica will eventually commit v within a constant number of rounds. □

Theorem 4. *Fides satisfies **Liveness** property.*

Proof. In Fides, assume a correct replica p_i receives transaction tx. Upon receiving tx, p_i creates a vertex v encapsulating tx along with potentially other transactions. The vertex v is then reliably broadcast to all other replicas.

By Lemma 18, every vertex v in the DAG is eventually committed by every correct replica. Since the transaction tx is contained in the vertex v, so it will also be committed by all correct replicas. Thus, Fides ensures that any transaction received by a correct replica will eventually be committed by all correct replicas, satisfying the **Liveness** property. □

E SGX Hardware vs. Simulation Overhead

To justify our use of SGX simulation mode (SGX-SIM) in geo-distributed WAN experiments, we conduct a systematic evaluation of the performance overhead introduced by real SGX hardware encryption (SGX-HW) compared to SGX-SIM in two direct measurement settings: local (no network) and LAN. We then use these measurements to interpret the WAN-scale setting used in the main evaluation.

E.1 Local Microbenchmark

We first isolate the raw overhead of enclave transitions and Memory Encryption Engine (MEE) paging by measuring individual ECALL latencies without any network involvement. Table 2 summarizes the results across four representative ECALL types.

Table 2: SGX-HW vs. SGX-SIM ECALL microbenchmark.

ECALL Type	SGX-SIM	SGX-HW	Overhead
Empty (transition only)	7.62 μs	10.46 μs	+2.84 μs (+37.2%)
Compute (4 KB payload)	10.68 μs	14.13 μs	+3.45 μs (+32.3%)
Memory copy (4 KB)	7.95 μs	11.11 μs	+3.16 μs (+39.7%)
Memory copy (64 KB)	17.64 μs	21.22 μs	+3.58 μs (+20.3%)

SGX-HW introduces a consistent overhead of approximately 3 μs per ECALL due to hardware-enforced security validations, Enclave Page Cache (EPC) lookups, and MEE encryption/decryption. The relative overhead exceeds 30% for small payloads but decreases for larger payloads as the computation itself begins to dominate.

E.2 LAN Benchmark

We next measure the end-to-end latency in a LAN setting, where a client on one machine issues requests to a server on another machine within the same datacenter. We use a 4 KB payload to match the microbenchmark compute workload.

Table 3: End-to-end latency in LAN (same datacenter).

Environment	End-to-End Latency (μs)
LAN + SGX-SIM	~93
LAN + SGX-HW	~93 (within noise)

Although a single ECALL adds ~3 μs in isolation, at LAN scale the enclave transition overlaps with concurrent socket writes and kernel scheduling rather than serializing on the critical path. The residual difference between SGX-HW and SGX-SIM falls within the run-to-run jitter (~1–5 μs) from userspace measurements, making the two statistically indistinguishable in this setting.

E.3 EPC Capacity and Enclave Memory Usage

The SGX-enabled `ecs.g7t.xlarge` instances used in our experiments provide 3.875 GiB of EPC. Each replica runs one enclave configured with an 8 MiB heap and two 4 MiB stacks, for a reserved budget of approximately 16 MiB per replica.

In our implementation, the enclave hosts only the minimal trusted components (*MC*, *RAC*, and *RNG*). Runtime profiling shows that the peak in-enclave heap usage is only 4096 B during our experiments. Consensus-level data structures such as the DAG, message queues, and transaction pools remain outside the enclave. As a result, the enclave memory footprint remains small and stable across the evaluated workloads, and none of our tested configurations approach EPC pressure or incur EPC paging.

E.4 Summary

These results confirm that SGX-HW overhead is only measurable in isolated, network-free microbenchmarks. Even in the most optimistic LAN setting, the overhead is already negligible (<0.1%). Under WAN-scale latencies (e.g., the 100 ms delays used in our DIN evaluation), the SGX-HW overhead becomes even more insignificant. This justifies our use of SGX-SIM for geo-distributed WAN experiments where SGX-enabled instances are not available across all cloud regions.